

Architecture Details

The model was implemented with a Multi-layer Perceptron architecture, with 3 hidden layers and a hidden size of 4096 neurons.

The general structure of each hidden layer is as follows:

1. Linear (nn.Linear)
2. Batch normalization (nn.BatchNorm1d)
3. GELU (Activation)
4. Dropout (Regularization)

Each hidden layer is being used as a fully connected layer. The batch normalisation serves to standardise the distributions of linear combinations in each layer, which stabilises training and also allows for a higher learning rate.

The activation function being used is the Gaussian Error Linear Unit (GELU). GELU is a variation of ReLU which mitigates the problem of vanishing gradients, by allowing gradients to retain a small magnitude rather than setting it to 0, as is the case in normal ReLU. In ReLU, everything smaller than zero is zero, but negative values in GELU are only partially suppressed.

Compared to Leaky ReLU, which uses a fixed slope for negative inputs, GELU provides a smooth and continuously varying response across the entire input range. This smoothness helps stabilize optimization and often leads to better performance, as it avoids the sharp activation boundaries present in ReLU-based functions.

$$GELU(x) = 0.5 * x * (1 + \tanh(\sqrt{\frac{2}{\pi}} * (x + 0.44715 * x^3)))$$

For each subsequent layer, the number of neurons are roughly halved, to fit the final dimensionality of the output of 50 classes. The dropout rates are also halved in this manner, based on a heuristic with the intuition that earlier layers are more unprocessed, and prone to noise, whereas deeper layers are heavily processed and should be relied on more greatly to make an accurate prediction.

For our model, the initial output size of the first layer is 4096, and the initial dropout is set to 0.5.

Training Methodology

Data processing

The training data was split into training and validation set by a 80/20 ratio.

Initialization

Xavier uniform initialization was used for the MLP's initial weights to prevent the gradients from vanishing or exploding in the initial stages of training. This weight initializer fills the weight tensors with random values in a uniform distribution defined as such:

$$W \sim U \left[-\frac{\sqrt{6}}{\sqrt{n_i+n_o}}, \frac{\sqrt{6}}{\sqrt{n_i+n_o}} \right]$$

where n_i is the number of input units (fan-in) and n_o is the number of output units (fan-out). Biases are initialized to 0.

Annealing & Learning Rate Scheduling

Cosine annealing adjusts the learning rate as training proceeds. The learning rate decreases following a cosine function, starting from a maximum value and gradually approaching a minimum value over a set number of training epochs or steps.

A warmup phase is used, where the learning rate slowly increases from a small initial value up to an initial maximum, stabilizing the early stages of training. We use 12 warmup epochs to avoid spikes and prevent early instability.

Random Search [SEE EXTRA NOTES AT PAGE 15]

The hyperparameter tuning process involved multiple extensive random searches over a wide range of configurations, ensuring that the final model was not the result of a narrow or biased search space. By exploring many combinations of learning rates, layer sizes, regularization strengths, and optimization settings, we increased the likelihood of discovering a near-optimal setup for this 50-class MLP. The consistency of the K-Fold validation results suggests that the selected hyperparameters generalize well and are not the product of overfitting to a particular split or trial. Overall, the breadth of the repeated random searches contributes to confidence that the model's performance represents a stable, well-supported optimum rather than a lucky configuration.

Loss Function

For this multi-class classification task (selecting one correct label out of 50), we use Cross Entropy Loss, which measures the divergence between the predicted probability distribution and the true distribution. PyTorch's `nn.CrossEntropyLoss` internally combines softmax with

negative log-likelihood, providing stable gradients for multi-class problems and encouraging higher confidence on correct predictions.

We apply label smoothing with `label_smoothing=0.15` to soften the target distribution. This reduces overconfidence, lowers variance, and generally improves the model's ability to generalize.

Regularization

Batch size: 512

A larger batch size helps reduce gradient noise, producing smoother, more stable updates and more consistent optimization across epochs.

Gaussian noise (`noise_std = 0.6`)

Adding Gaussian noise to the inputs acts as a strong regularizer. It perturbs the data during training, making the model more robust and helping prevent overfitting.

Submission

We used to get 81.93% validation accuracy without the `label_smoothing` and `max_norm` changes. We were able to squeeze out +0.6-0.7% validation accuracy by setting the Cross Entropy loss's `label_smoothing=0.15`, and being less strict with gradient clipping using `max_norm=10.0` instead of `max_norm=1.0`.

We retrained for 125 epochs on the full training dataset for a small boost in accuracy (1-2%). Kaggle submission gave 0.839 -> 83.9% accuracy on the first half of the test dataset, while 80/20 train-val split gave 82.65% validation accuracy at epoch 125.

It's possible that there exists a better set of hyperparameters that could give higher accuracies.

Optimization

Optimizer

The AdamW optimizer was used with L2 regularization (weight decay=0.05) and an initial learning rate of 0.005. AdamW was used over Adam as it applies weight decay as a separate step after the gradient update. AdamW uses mini-batch descent, to help it get past local minima. It also incorporates a momentum term, which is essentially an exponentially weighted moving average of the past gradients, to retain some of the magnitude from previous updates, and this can also help training with escaping local minima. Overall, we used AdamW optimizer to improve generalization and to avoid memorizing training data.

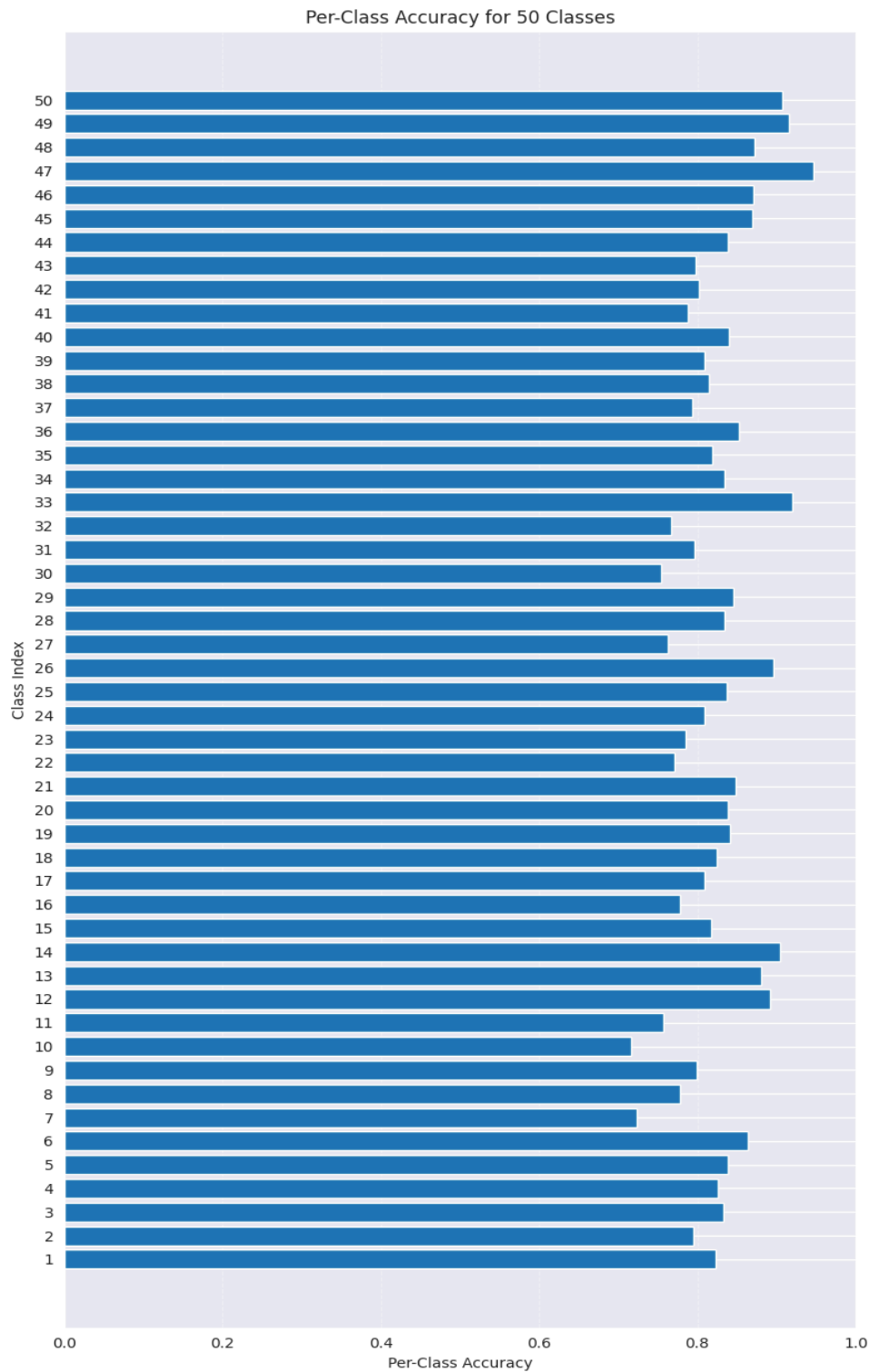
Gradient Clipping

We apply gradient clipping with $\text{max_norm} = 10.0$ to prevent exploding gradients. This technique constrains the overall gradient norm during backpropagation, ensuring stable updates and improving training reliability, especially in deeper or more sensitive architectures.

1. Metrics:

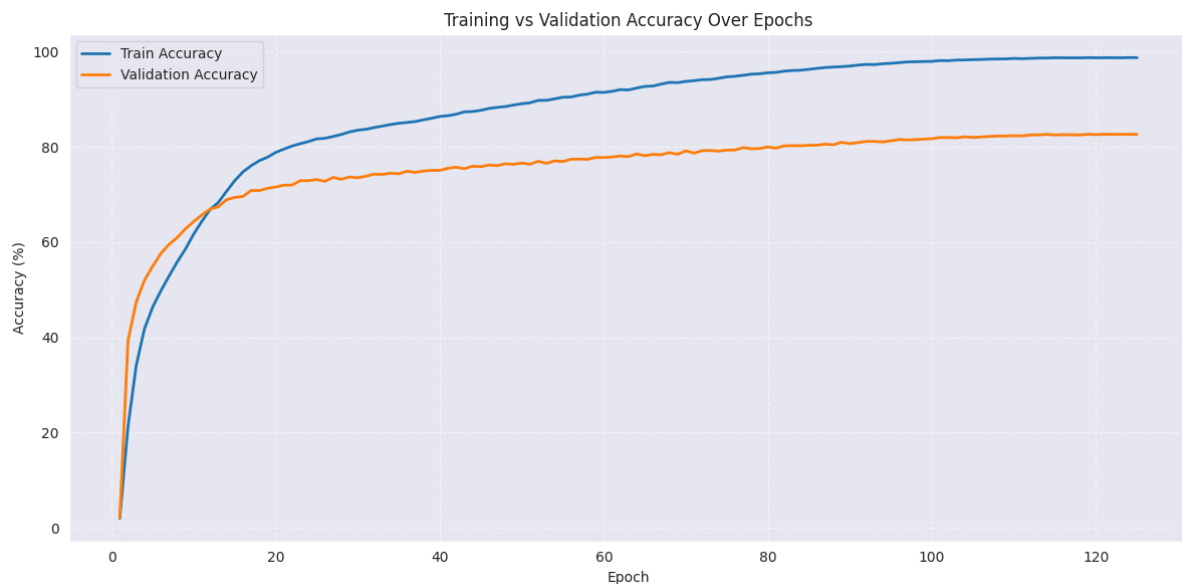
From this section and later, the graphs refer to the evaluation on the 20% validation set.

Per-Class Accuracy

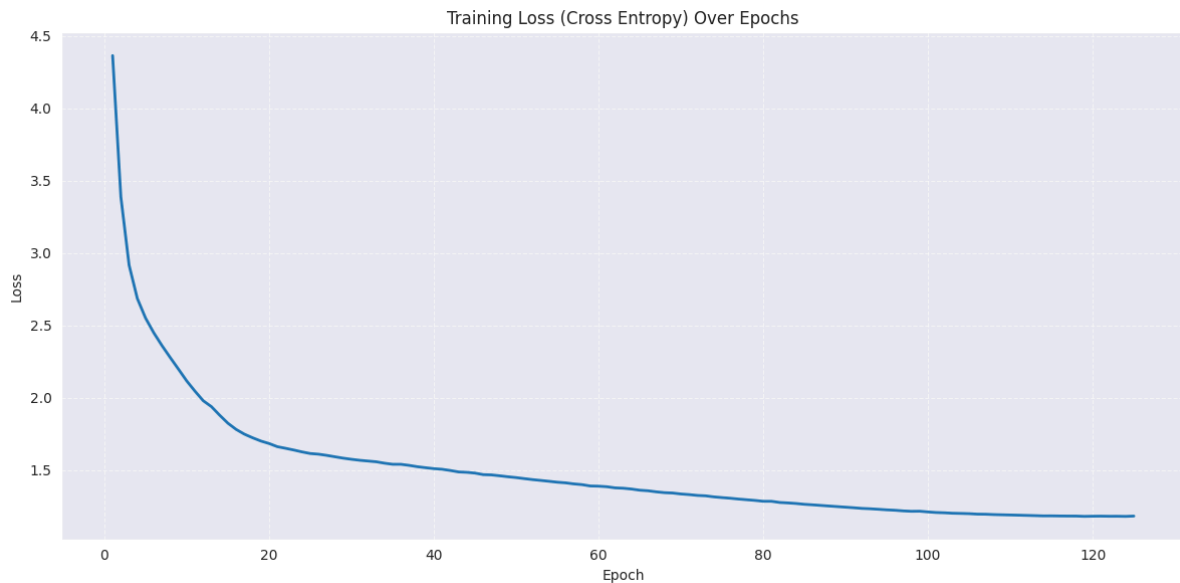


This plot shows the per-class accuracy across 50 classes, revealing how consistently the model performs on each individual category. The accuracies are generally high and fairly uniform, with most classes falling in the 0.75-0.90 range, indicating that the model does not rely heavily on just a few easy classes. A few classes achieve slightly higher accuracy near the top end, suggesting they are easier for the model to distinguish, while a small number near the lower end show room for improvement. The distribution suggests strong and balanced class-level performance, without severe class-specific weaknesses.

Accuracy and Loss Graphs



This graph shows the training and validation accuracy of an MLP model trained to classify 50 object categories. Training accuracy rises quickly and eventually approaches nearly 100%, indicating that the network is able to fit the training data extremely well. In contrast, validation accuracy plateaus around the low-80% range, showing a clear generalization gap. This pattern suggests that while the MLP is powerful enough to memorize the training set, it struggles to generalize perfectly to unseen examples, due to the complexity of the 50-class task. The slow, steady improvement in validation accuracy across epochs indicates stable training but also hints at mild overfitting.



This plot shows the training loss of the MLP steadily decreasing over the course of training while classifying 50 object categories. The sharp drop in the early epochs indicates that the model quickly learns coarse decision boundaries, and the slower, gradual decline afterward reflects fine-tuning of class separation. By the end of training, the curve flattens out, suggesting the model has reached a point of diminishing returns where additional epochs yield only marginal improvements. Combined with the earlier accuracy plot, this behavior is consistent with an MLP that fits the training data well but may still struggle to generalize perfectly due to the complexity of the 50-class task.

K-Fold Validation

Fold 1 Best Validation Accuracy: 82.10%
Fold 2 Best Validation Accuracy: 82.05%
Fold 3 Best Validation Accuracy: 82.20%
Fold 4 Best Validation Accuracy: 82.90%
Fold 5 Best Validation Accuracy: 82.65%
K-Fold Validation Results (5 folds): Mean = 82.38%, Std = 0.38%

These K-Fold validation results show that the MLP performs consistently across different splits of the dataset when classifying 50 object categories. All five folds achieve validation accuracies tightly clustered around ~82%, with a very small standard deviation of 0.38%. This low variance indicates that the model's performance is stable and not overly dependent on a particular train-validation split, suggesting good robustness in how it learns the overall data distribution. At the same time, the mean accuracy of 82.38% aligns with the plateau observed in the validation curve, reinforcing that this is likely the model's practical generalization limit given its architecture and the complexity of the task.

2. Comparative insights:

For this part, Item Index or Class Index has the same meaning as Class Number. We decided to 1-index the class numbers, so the class labels in the training data's CSV are from 0-49, while the 1-indexed classes are from 1-50.

Precision/Recall/F1-Scores

Using the values from scikit-learn's classification report, we generated these plots:



Average Precision = 0.8293747145122006

Average Recall = 0.8266521316982987

Average F1-Score = 0.827058451516312

For Average Precision we have the correct rate around 82.937%, Classes with high precision are class 6,12,14,29,49 around 90%+, which means our MLP model is correct when assigning these labels. Class 24 has the lowest Precision score.

$$Precision = \frac{TP}{TP+FP}$$

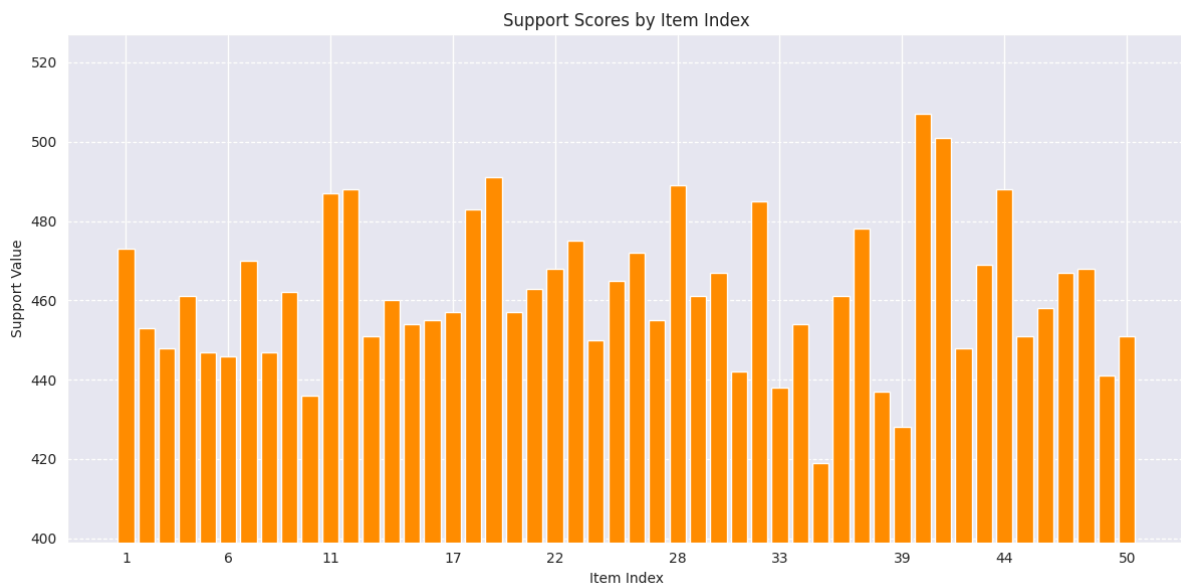
For Average Recall we have the correct rate around 82.66%, which means that our MLP model out of class 50, our model has an average rate of 82.66% that takes all samples that actually belong to the correct class.

$$Recall = \frac{TP}{TP+FN}$$

For Average F1-score which balances both precision and recall, we have a relatively high F1 score on classes 14,47,49 .

$$F1 = \frac{2 * TP}{2 * TP + FP + FN}$$

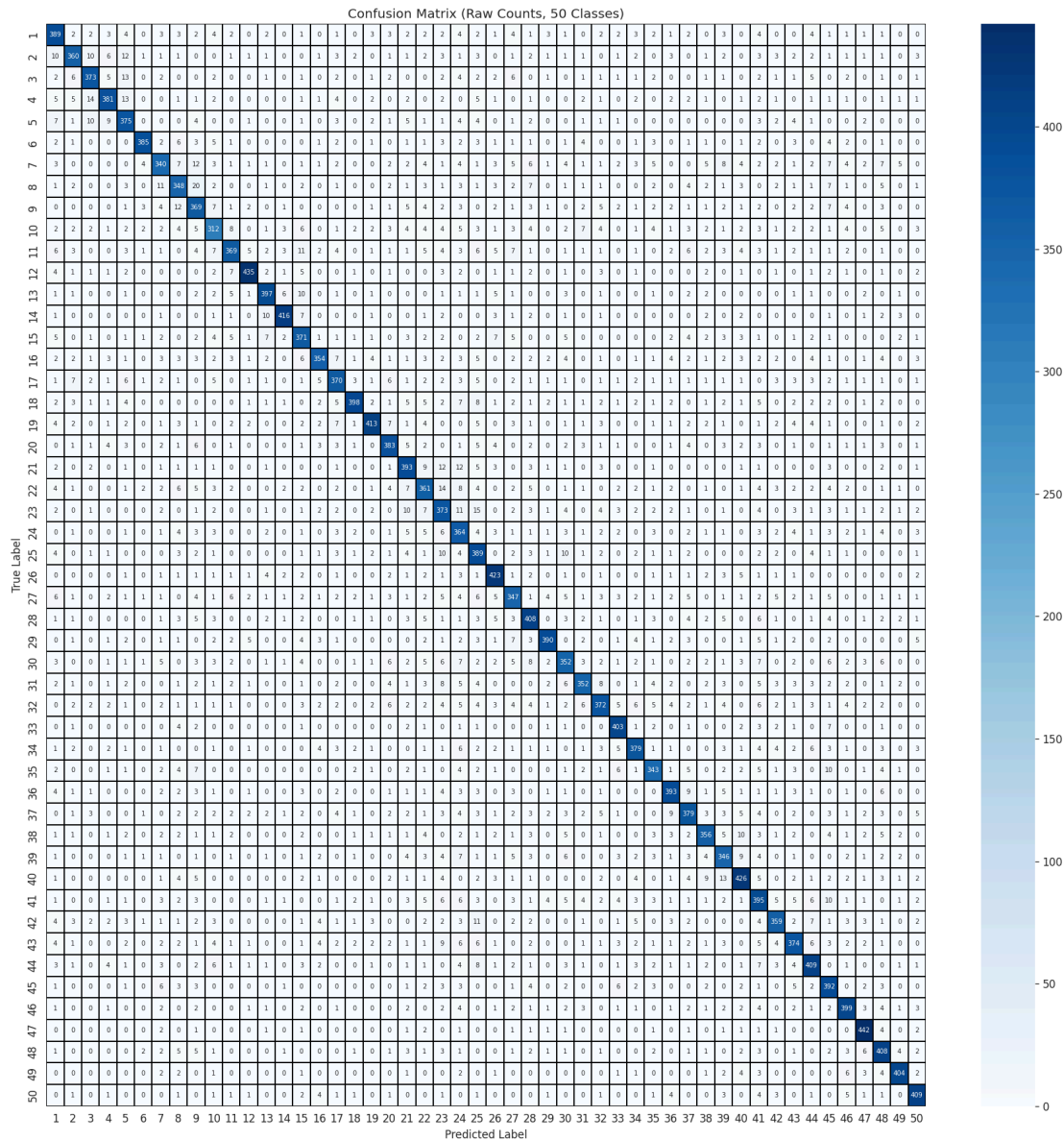
Support Scores



This chart shows the support scores for each of the 50 object classes based on the validation set, effectively indicating how many samples from each class were present during evaluation. The bars are fairly uniform, with most classes having support values in a similar range, typically between about 440 and 500 samples. This even distribution suggests that the validation set is well-balanced across all 50 categories, which is important for obtaining reliable accuracy metrics. A few classes have slightly higher or lower support, but not enough to skew the results.

3. Error analysis:

Confusion Matrix (This one needs A LOT of zooming in)



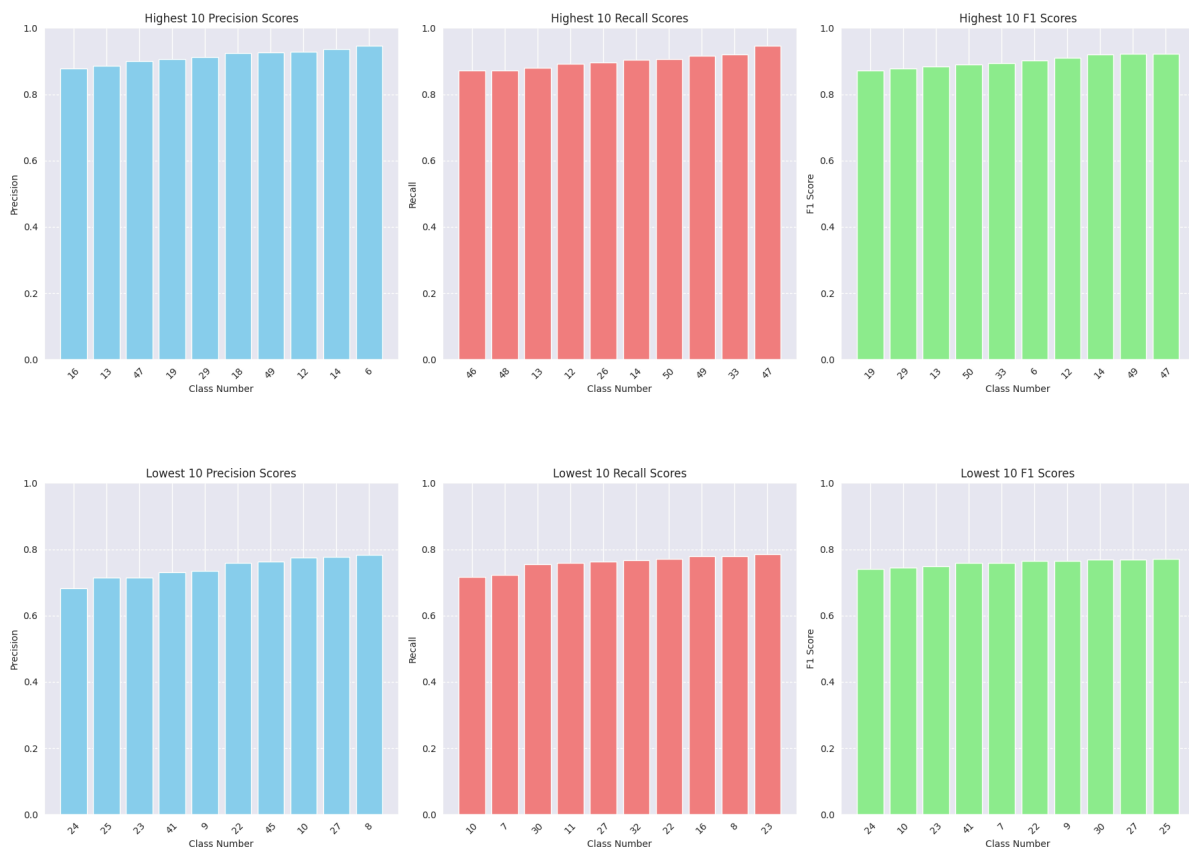
This confusion matrix shows strong, concentrated performance along the diagonal, meaning the model correctly predicts most samples for nearly every class. The diagonal cells are consistently dark, indicating high true-positive counts, while off-diagonal values remain very small, showing that misclassifications are infrequent and broadly distributed rather than dominated by specific class pairs.

There are no obvious clusters of confusion where certain groups of classes are frequently mistaken for one another. Instead, errors appear as small, isolated counts scattered across the matrix. The pattern reflects a well-calibrated classifier with clear class boundaries, consistent with the high per-class accuracies observed earlier.

From confusion Matrix: We observed that most misclassifications occur in the off diagonal cells that are close to the main diagonals. True label 7, predicted label 46 that there are some off-diagonal misclassifications that may be due to two classes having overlapping feature distributions that MLP architecture cannot separate them perfectly . This means in for 500 features dimensional space, clusters overlap. On the other hand, for example , if the True label is 8 and 20 of them predicted labels are 9, we have a confusion that our model tends to confuse adjacent classes rather than distant classes. This shows our MLP model learned continuous decision boundaries that lead to local misclassifications.

4. Strong classes:

Top-10 Highest and Lowest Precision/Recall/F1-Scores



These figures show the Highest and Lowest Precision scores, recall scores and F1 scores.

The lowest precision are class 7, 8, 9, 10, 22, 23, 24, 25, 27 which means our MLP model incorrectly predicts these classes frequently. (with high false positives).

From the highest Precision, highest recall scores and highest F1 scores, the strongest classes are: 6, 12, 13, 14, 33, 47, 49, which means our MLP model correctly predicts these

classes often, which makes these classes the strongest classes. This means these classes have few false positives (high precision) and few false negatives (high recall scores) which lead to high F1 scores.

5. Depth and Size Effects:

Model	Variant	Accuracy	Precision	Recall	F1-Measure
Multiple Layer Perceptron	2-layer MLP(h=512)	55.91% (early stopped)	56.70%	55.99%	55.88%
	4-layer MLP(h=512)	51.59% (early stopped)	53.41%	51.70%	51.66%
	3-layer MLP(h=512)	53.60% (early stopped)	55.24%	53.69%	53.73%
	3-layer MLP(h=1024)	61.71% (early stopped)	63.14%	61.74%	62.00%
	3-layer MLP(h=2048)	79.24%	79.71%	79.27%	79.35%
	3-layer MLP(h=4096)	82.65%	82.93%	82.66%	82.70%

* h indicates the number of neurons for the the first input layer

We applied early stopping if validation accuracy does not improve for 10 epochs OR validation accuracy is under 65% at epoch 20.

For our team to understand how network capacity affects the performance of classification, we evaluated several MLP architectures by varying depths and hidden layer sizes:

- 2-layer MLPs(two hidden layers)
- 3-layer MLPs(three hidden layers)
- 4-layer MLPs (four hidden layers)
- With different hidden size from 512 to 4096

As more layers are added, the training loss drops a lot for the deeper models, so we can conclude that a more layered MLP fits the training data more tightly. We know that deeper layers increase model capacity, allowing the model to learn more complex decision boundaries. However, comparing the 2,3, and 4-layer MLP models with fixed h=512. the test

accuracy does not improve, and actually becomes slightly lower as more layers are added. For the 500 features, a 3-layer MLP is enough; adding more depth mainly helps fit the training data but not the test data. This means the depth improves training loss but not test performance. After about 80 epochs, the accuracy also stops increasing much.

The test accuracy improves then drops when hidden size increases, when the accuracy is high, and after extra training, the model starts fitting noise and not patterns. This means the model underfit first, then reaches its peak, then overfit with more epochs. This leads to overfitting because the training loss drops as size grows. So, it means that when the hidden size varies, and the MLP trains on 500 features, the model at some point begins to overfit as the training loss decreases. Medium sizes give best output accuracy (for example, 2048 hidden size and 1024 hidden size).

6. Summary:

All models were trained under the same conditions (AdamW optimizer, Xavier initialization, cosine LR schedule, Gaussian noise, batch size 512) to make sure to measure the difference in performance.

Then we came up with the best performing architecture with hidden layer size from 4096→2048→1024→Output, which is a funnel-shaped architecture where each subsequent layer becomes narrower. The reasons we learned from this structure is that each layer progressively condenses information and by shrinking the size to avoid overfitting, memorize noise and learn redundant features. After comparing architectures of MLPs with different depths and sizes, the funnel shaped 3 layer MLP produces the strongest test performance.

The results of metric distributions match mostly with the confusion matrix patterns. Which strong classes are separable and distinctive that makes accurate predictions and weak classes overlap feature patterns with more misclassifications.

Links

Instructions:

<https://github.com/COMP432/.instructions>

Github code:

<https://github.com/COMP432/Team-26>

Xavier initialization:

https://docs.pytorch.org/docs/stable/nn.init.html#torch.nn.init.xavier_uniform

GeLU:

<https://docs.pytorch.org/docs/stable/generated/torch.nn.GELU.html>

Precision, Recall, Accuracy

<https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall>

F1-Score

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html

Extra Notes

[4th Feb 2025]: When my team was trying to get this report out, we only gave a general description of the hyperparameter tuning process, which led us to losing some marks there. I would like to rectify this here with what I remember now.

[illegible]

We adopted a staged hyperparameter tuning strategy that progressed from broad exploration to focused refinement.

We began by fixing the number of neurons per hidden layer at 512 and experimented with different network depths. After evaluating multiple configurations, we found that a 3-hidden-layer MLP offered the best trade-off between capacity and stability and used this architecture as the baseline for further tuning.

With this architecture fixed, we conducted extensive random searches over a wide hyperparameter space, varying learning rates, regularization strengths, optimizer settings, and related training parameters. The goal at this stage was broad exploration rather than fine optimization. Through this process, we identified several configurations achieving approximately 70% validation accuracy, indicating that the model capacity was appropriate and that performance was not driven by a narrow or lucky configuration.

After identifying promising regions of the search space, we analyzed training and validation plots (loss and accuracy curves) to diagnose underfitting, overfitting, and learning dynamics. Guided by these observations, we performed subsequent random searches over progressively narrower ranges of the most influential hyperparameters, focusing on stabilizing training and improving generalization.

Finally, to extract additional performance gains, we conducted a selective grid search over a tightly constrained subset of hyperparameters centered around the best-performing configurations discovered during random search. This final step was used to fine-tune values rather than explore new regions of the space.

Throughout tuning, K-Fold cross-validation was used to evaluate candidate configurations. The consistency of performance across folds suggests that the selected hyperparameters generalize well and that the final model performance is not the result of overfitting to a particular split or trial.